

ĐỀ THI ĐỀ XUẤT – LẬP TRÌNH NÂNG CAO

Phạm vi: Kiến thức OOP, Design Pattern cơ bản & Tích hợp/Triển khai (CI/CD)

(Thời gian làm bài: 90 phút)

Lưu ý: Đây là đề thi được thiết kế dựa trên cấu trúc đề mẫu và bài học về Tích hợp, Triển khai. Đề thi bao gồm cả phần trắc nghiệm và tự luận.

BÀI 1. TRẮC NGHIỆM (5 ĐIỂM)

Mỗi câu 0.5 điểm. Hãy chọn một đáp án đúng nhất.

Câu 1. Cho đoạn mã Java sau:

```
class Parent {
    public static void display() { System.out.print("Parent "); }
}
class Child extends Parent {
    public static void display() { System.out.print("Child "); }
}
public class Main {
    public static void main(String[] args) {
        Parent obj = new Child();
        obj.display();
    }
}
```

Kết quả in ra màn hình của chương trình là gì?

- A. Child
B. Parent
C. Lỗi biên dịch do phương thức static không thể bị ghi đè (override).
D. Lỗi Runtime.

Câu 2. Giả sử bạn có interface Printable và một lớp Generic khai báo như sau: `class Storage<T> extends Printable { ... }`. Dòng mã nào sau đây sẽ gây lỗi biên dịch?

- A. `Storage<Printable> s = new Storage<>();`
B. `Storage<String> s = new Storage<>();`
C. `Storage<Report> s = new Storage<>();` (Biết rằng Report implements Printable)
D. Cả A và B đều gây lỗi.

Câu 3. Trong môi trường đa luồng (multithreading), điểm khác biệt quan trọng nhất giữa `Callable<T>` và `Runnable` là gì?

- A. `Callable` bắt buộc phải tạo Thread thủ công, trong khi `Runnable` chạy qua `ExecutorService`.
- B. `Callable` có thể trả về một kết quả thông qua `Future` và ném ra `Checked Exception`, `Runnable` thì không.
- C. `Callable` đảm bảo không bao giờ bị `Deadlock`.
- D. Hàm `call()` của `Callable` không làm block luồng chính (Main thread) khi gọi `Future.get()`.

Câu 4. Khi thực thi lệnh `mvn package` trong Maven, nếu giai đoạn chạy Unit Test (phase: `test`) gặp lỗi (Fail), điều gì sẽ xảy ra?

- A. Maven bỏ qua lỗi, tiếp tục biên dịch và tạo ra file `.jar`.
- B. Maven dừng ngay lập tức quá trình build, không có file `.jar` nào được tạo ra.
- C. Maven tạo ra file `.jar` nhưng thêm hậu tố `-failed` vào tên file.
- D. Maven tự động chuyển sang cấu hình debug để chạy lại test.

Câu 5. Trong file `pom.xml`, khi khai báo một thư viện (ví dụ: MySQL JDBC Driver) không cần thiết trong lúc biên dịch code (compile) nhưng bắt buộc phải có môi trường lúc chương trình chạy, bạn nên đặt thẻ `<scope>` là gì?

- A. `compile`
- B. `provided`
- C. `test`
- D. `runtime`

Câu 6. Hệ thống logging (Logback/SLF4J) đang được cấu hình root level là: `<root level="WARN">`. Khẳng định nào sau đây là ĐÚNG về đoạn mã dưới đây?

```
logger.debug("Tải dữ liệu...");
logger.info("Kết nối DB thành công.");
logger.warn("Sắp hết dung lượng đĩa.");
logger.error("Lỗi crash ứng dụng.");
```

- A. Tất cả các dòng log đều được ghi lại.
- B. Chỉ có dòng `WARN` được ghi lại.
- C. Chỉ có dòng `WARN` và `ERROR` được ghi lại.
- D. Chỉ có dòng `DEBUG` và `INFO` được ghi lại.

Câu 7. Lý do chính khiến Parameterized Logging (Sử dụng { }) được khuyến khích so với việc nối chuỗi truyền thống (String Concatenation) là gì?

- A. Mã hóa thông tin an toàn hơn, tránh lộ dữ liệu nhạy cảm.
- B. Tiết kiệm hiệu năng (chi phí nối chuỗi) khi log level hiện tại đang tắt, vì đối tượng chỉ bị đánh giá `toString()` khi log thực sự được in ra.
- C. Bắt buộc phải dùng để tích hợp với Checkstyle.
- D. Ngăn ngừa lỗi biên dịch nếu các biến truyền vào là null.

Câu 8. Khi sử dụng Checkstyle để phân tích mã nguồn (Static Code Analysis), khái niệm độ phức tạp vòng (Cyclomatic Complexity) dùng để đo lường điều gì?

- A. Số lượng các thư viện (dependencies) vòng lặp lẫn nhau trong file `pom.xml`.
- B. Thời gian thực thi (Big-O) của thuật toán.
- C. Số lượng các đường dẫn (paths) thực thi độc lập qua mã nguồn (phụ thuộc vào số lượng câu lệnh `if/else`, `for`, `switch...`).
- D. Độ dài của một class (tổng số dòng code).

Câu 9. Trong file cấu hình GitHub Actions (YAML), tính năng cấu hình `matrix` (Ví dụ: `os: [ubuntu-latest, windows-latest, macos-latest]`) có mục đích chính là gì?

- A. Tạo ra một ma trận bảo mật mã hóa code.
- B. Sinh ra nhiều luồng (Jobs) chạy song song thử nghiệm trên nhiều môi trường hệ điều hành khác nhau.
- C. Tự động lưu cache (Caching) ở nhiều khu vực địa lý khác nhau.
- D. Giới hạn số người được phép Pull Request vào nhánh chính.

Câu 10. Trong luồng CI/CD với GitHub Actions, sau khi lệnh `mvn package` chạy thành công, máy ảo (Runner) sẽ bị hủy. Để giữ lại và cho phép tải xuống file `.jar` kết quả, bạn cần sử dụng action nào?

- A. `actions/checkout`
- B. `actions/cache`
- C. `actions/upload-artifact`
- D. `actions/download-jar`

BÀI 2. THIẾT KẾ HƯỚNG ĐỐI TƯỢNG (2.5 ĐIỂM)

Một hệ thống Quản lý Khóa học Trực tuyến (E-Learning) có các nhóm người dùng gồm: **Học viên** và **Giảng viên**. Khóa học được tạo ra bởi Giảng viên và chứa nhiều **Tài nguyên học tập** (Bài học).

Tài nguyên học tập hiện tại gồm 3 loại: *Video*, *Tài liệu đọc (PDF)*, và *Bài kiểm tra (Quiz)*. Mỗi tài nguyên học tập đều có tiêu đề (title), ID và thời lượng ước tính.

Có những tài nguyên cho phép "Học thử miễn phí" (Free preview), trong khi các tài nguyên khác yêu cầu học viên phải "Đăng ký khóa học" (Premium) mới được truy cập nội dung.
Hệ thống cần theo dõi tiến độ: ghi nhận xem Học viên đã hoàn thành tài nguyên nào.

Yêu cầu:

1. Xác định các lớp (Class) chính, các thuộc tính cơ bản và mối quan hệ giữa các lớp. (1.0 điểm)
2. Chỉ rõ việc áp dụng Abstract Class và Interface ở đâu cho hợp lý. (0.5 điểm)
3. Trình bày biểu đồ lớp mức khái quát (Có thể biểu diễn bằng giả mã code Java hoặc cấu trúc text rõ ràng). (0.5 điểm)
4. Nếu trong tương lai hệ thống muốn bổ sung thêm một loại tài nguyên mới là "Môi trường lập trình tương tác" (Interactive Coding) mà **không làm ảnh hưởng (sửa đổi) đến các file code đang chạy ổn định**, thiết kế ở câu 2 và 3 của bạn đã đáp ứng được nguyên lý *Open/Closed Principle (OCP)* như thế nào? Hãy giải thích. (0.5 điểm)

BÀI 3. KIỂM THỬ PHẦN MỀM (1 ĐIỂM)

Trong hệ thống CI Pipeline, một hàm kiểm tra trạng thái hợp lệ của một lần build (Test Quality Gate) được viết như sau:

```
boolean checkQualityGate(int totalTests, int failedTests, double coverage)
```

Quy tắc kinh doanh (Business Rules):

- Lần build được đánh giá là Đạt (Pass) nếu: Không có test nào rớt (`failedTests == 0`) VÀ độ bao phủ mã nguồn (`coverage`) đạt tối thiểu `80.0%`.
- Dữ liệu đầu vào bị coi là KHÔNG HỢP LỆ (ném ra `IllegalArgumentException`) nếu:
 - `totalTests <= 0`
 - `failedTests < 0` hoặc `failedTests > totalTests`
 - `coverage < 0.0` hoặc `coverage > 100.0`

Yêu cầu: Sinh bộ Test Case (các trường hợp kiểm thử) bằng kỹ thuật **Phân tích giá trị biên (Boundary Value Analysis)** tập trung kiểm tra ranh giới cho tham số `coverage` và tham số `failedTests`.

BÀI 4. CLEAN CODE VÀ REFACTORING (1.5 ĐIỂM)

Cho đoạn mã chịu trách nhiệm gửi thông báo (Notification) sau khi hệ thống CI/CD triển khai xong như sau:

```
class DeploymentNotifier {  
    public void notify(String platform, String message) {
```

```
        if (platform.equals("EMAIL")) {
            System.out.println("Kết nối tới SMTP Server...");
            System.out.println("Gửi Email: " + message);
        } else if (platform.equals("SLACK")) {
            System.out.println("Kết nối tới Slack API bằng Token...");
            System.out.println("Gửi tin nhắn Slack: " + message);
        } else if (platform.equals("TELEGRAM")) {
            System.out.println("Khởi tạo Telegram Bot...");
            System.out.println("Gửi Telegram: " + message);
        } else {
            System.out.println("Không hỗ trợ nền tảng này!");
        }
    }
}
```

Yêu cầu:

1. Chỉ ra vấn đề thiết kế (Code Smell) đang tồn tại trong đoạn mã trên. Nhược điểm của cách viết này khi duy trì trong thực tế là gì? (0.5 điểm)
2. Đề xuất cách tái cấu trúc (Refactoring) đoạn mã trên để cải thiện chất lượng thiết kế. Bạn hãy viết lại (giả mã / Java code) cấu trúc mới theo hướng dễ dàng mở rộng (Ví dụ: sau này công ty muốn gửi cảnh báo qua *Discord*, *Zalo*...). (1.0 điểm)